Discover Computing

RESEARCH Open Access

# Computation offloading and association in MEC-assisted V2X network for delay minimization and collision avoidance via deep Q-network

Xinmin Li[1*], Yifan Pu[1], Chenwen Yan[2], Wenwen Duan[2] and Xuhao Zhang[2,3]

*Correspondence:
Xinmin Li
lixinmin@cdu.edu.cn
[1]College of Computer Science, Chengdu University, Chengdu 610100, China
[2]School of Information and Control Engineering, Southwest University of Science and Technology, Mianyang 621000, China
[3]Sichuan Energy Internet Research Institute, Chengdu 610213, China

**Abstract**
The task processing delay and road safety are the key challenges of the vehicle-to-everything (V2X) network in the sixth generation system. By offloading the computation-intensive tasks of the vehicles to road side unit (RSU) and base station (BS), mobile edge computing (MEC) technology can reduce the processing delay of V2X network. However, it is difficult to dynamically associate the moving vehicles to MEC servers and offload the tasks, especially in the low collision scenario. Thus, we consider the MEC-assisted multi-vehicle V2X system, where the vehicles can offload the computation-intensive tasks to the MEC servers deployed at the multi-antenna RSUs and BS with the zero-forcing receivers. The system delay minimization problem is formulated under the delay and collision constraints to satisfy the task processing delay and safety requirements in the V2X system. Due to the coupling of the association, offloading ratio and driving acceleration, the system delay minimization problem is difficult to solve. Thus, the intelligent scheme based on deep Q-network is proposed to jointly optimize the association, offloading ratio, driving acceleration. The piecewise reward function is designed depending on the delay, energy and collision constraints, while the proposed algorithm trains the vehicles to obtain superior actions consisting of MEC server association, offloading ratio and driving acceleration. Simulation results show that the system delay of the proposed algorithm can reduce by 16.44% and 26.64% compared with Q-learning and local computing schemes, respectively. Under different road length and number of vehicles settings, the proposed algorithm can also outperform the benchmark schemes.

**Keywords** Vehicle-to-everything, Mobile edge computing, System delay, Deep Q-network, Vehicle collision

## 1 Introduction

The sixth generation (6G) wireless communication system is expected to achieve a hundredfold increase in transmission capacity and reduce the network latency from milliseconds to microseconds, which enables the large-scale intelligent interconnection

among the humans, devices and virtual entities [1]. Thus, 6G system integrates multiple advanced communication technologies including terahertz communication, unmanned aerial vehicle (UAV) communication and vehicle-to-everything (V2X) network, to form a high-speed, low-latency and wide-coverage wireless network. As a typical application scenario of 6G system, V2X technology enables the real-time perception and exchange of the road information through vehicle-to-vehicle (V2V), vehicle-to-infrastructure (V2I), and vehicle-to-pedestrian (V2P) communication links [1]. It is expected to significantly enhance the safety, communication efficiency, and connectivity of vehicular systems [2]. To mitigate the road accidents such as vehicle collisions, human injuries and property damage, the third generation partnership project (3GPP) specifies proposed the latency requirements: the maximum latency for precrash sensing and emergency vehicle warning is limited to 20 ms and 100 ms, respectively [3]. However, the intelligent V2X applications such as autonomous driving, trajectory planning and virtual reality, involve the computation-intensive and latency-sensitive tasks. Due to the limited computing and storage resources [4, 5], the vehicles are unable to efficiently process these tasks locally.

Mobile edge computing (MEC) enhances the computing and storage capabilities of V2X systems by offloading the tasks to the edge servers, such as base stations (BSs) and road side units (RSUs), thereby effectively reducing end-to-end (E2E) latency [6, 7]. Recently, diverse architectures and optimization strategies have been explored to further reduce E2E delay. For instance, a MEC integrated framework was proposed in [8] to minimize E2E latency from the radio, backhaul and processing perspectives and obtained the improvement under the high traffic density. Meanwhile, [9] studied the mode selection and resource allocation in MEC-assisted V2X networks under energy and bandwidth constraints, in which the mixed-integer nonlinear problem was solved via alternating convex optimization. In addition, standardization-oriented researcher has investigated the roadside MEC deployments with the sensor fusion, to enhance the scalability for the real-world V2X applications [10].

In the recent years, extensive studies have focused on the joint optimization of task offloading and resource allocation in MEC system. For example, the distributed task offloading frameworks have been proposed to reduce the BS computational load [11] and the game-theoretic models have been studied to minimize the energy consumption under the delay and vehicle utility constraints [12]. Moreover, the cooperative offloading scheme has been designed to decrease the communication distances and reduce the task latency [13]. Resource allocation plays a critical role in the joint optimization of the task offloading and limited network resources [14]. In multi-user collaborative MEC networks, tasks can be computed locally or offloaded to neighboring devices or processed by MEC servers. In [15], a long-term energy-constrained optimization method was proposed to minimize the overall energy consumption by jointly optimizing offloading and resource allocation. Considering the vehicle mobility and the latency constraints, authors in [16] formulated a joint optimization model to minimize both energy consumption and service delay, while the authors in[17] developed a two-stage heuristic algorithm to achieve te energy-efficient resource allocation in multi-user and multi-edge servers system.

Nevertheless, only reducing communication latency can not guarantee the road safety. According to the World Health Organization (WHO), approximately 2.4 people are killed in traffic accidents every minute and the number of accidents continues

to rise [18]. V2X systems aim to enhance the traffic safety through trajectory design, driving strategy optimization and the cooperative lane changing. Trajectory prediction serves as the core of collision warning systems, it yet remains challenging to ensure the real-time accuracy due to the transmission delay, packet loss, and limited onboard computation. To address this issue, [19] proposed a trajectory prediction and collision model based on the pruned state-refinement long short-term memory (LSTM) network, which maintains the high accuracy and early warning capability under the resource constraints. Cooperative collision avoidance schemes based on V2V communication were developed in [20], while the authors in [21] studied the collision-avoidance systems for the vulnerable road users such as the pedestrians and cyclists. Furthermore, [22] designed a decision-making strategy based on Proximal Policy Optimization (PPO) to balance the traffic efficiency and safety while reducing the discomfort experience caused by the sudden acceleration or deceleration.

As 6G networks evolve toward the intelligent and autonomous operation, leveraging deep learning and reinforcement learning for the dynamic decision-making has become an emerging trend. Deep reinforcement learning (DRL) provides a powerful framework for learning optimal strategies in the non-convex and complex environments [23, 24]. In [25], a multi-agent DRL algorithm was proposed to jointly optimize the server association, transmit power and offloading ratio in the V2X networks to achieve the balance between delay and energy consumption. In [26], a DRL approach based on Proximal Policy Optimization was used to improve traffic efficiency, and authors in [27] proposed a DRL-based autonomous driving algorithm to optimize the speed and lane changing to enhance the traffic throughput and avoid the collisions.

While the recent studies have applied DRL to enhance task offloading and driving control in V2X networks, most of them still focus on the fixed driving scenarios [15–17] [25–28]. These approaches neglect the dynamics of the driving acceleration and collision, which limits their application in realistic multi-vehicle scenarios. In this work, an intelligent joint optimization of MEC server association, task offloading ratio, and driving acceleration is formulated under energy, delay, and collision constraints, to achieve the balance between communication efficiency and driving safety. Moreover, unlike Q-learning [12] and the cooperative offloading schemes [13] that relied on fixed rewards and decoupled decision processes, the proposed DQN-based method integrated the communication and driving control via a piecewise reward design, enabling the vehicles to autonomously learn the adaptive delay–energy–safety strategies, which effectively minimizes the system delay while ensuring the driving safety by the task offloading and vehicle trajectory decisions. The main contributions of this work are summarized as follows:

- An uplink MEC-assisted V2X system is considered, where the multi-antenna BS and multiple RSUs with zero-forcing (ZF) receivers jointly equip the MEC servers to offload the intensive task of the vehicles. The system delay minimization problem is formulated under the energy, speed and collision constraints to analyze the joint effects of the association, offloading and acceleration.
- To handle the non-convex coupling among these variables, DQN-based intelligent algorithm is proposed by designing the joint action space by combing the offloading ratio, server association and driving acceleration. The specific piecewise reward

function related to the processing delay, energy consumption, and collisions to satisfy the corresponding constraints.

- Extensive simulations validate the effectiveness of the proposed scheme under different packet size and road length settings. It can reduce the system delay by 23.61% and 33.20% compared with Q-learning and local computing schemes, respectively, and achieves superior performance under different road lengths, vehicle densities, and packet sizes.

## 2 System model

As depicted in Fig. 1, the MEC-assisted multi-vehicle V2X system is considered in this work, which consists of one BS, $L$ RSUs and $K$ single-antenna vehicles. The length and width of the road and vehicles are $L_{\mathrm{rl}}$, $L_{\mathrm{rw}}$, $L_{\mathrm{vl}}$ and $L_{\mathrm{vw}}$, respectively. Due to the limited processing capability of vehicles, the tasks only can be offloaded to the MEC servers deployed at the BS or RSUs depending on the association. The models of the computation offloading consist of the full offloading and partial offloading [29]. The full offloading model offloads the whole computation task to the MEC server while the partial offloading model flexibly offloads the partial task to the MEC server and other tasks are computed locally. Thus, the partial offloading model is adopted in this work, in which the vehicles can compute the partial task locally or offload the task to the $N^{\mathrm{R}}$-antenna RSU via V2I link and $N^{\mathrm{B}}$-antenna BS via the vehicle-to-base station (V2B) communication link for the edge computing, respectively.

### 2.1 Trajectory and collision model

#### *2.1.1 Trajectory model*

In Fig. 1, there are $K$ moving vehicles driving in four directions, i.e., east, west, north and south. Thus, the direction of the $k$-th vehicle at the time slot $t$ is

$$\theta_{k,t} = \begin{cases} 0, & \text{if driving east,} \\ \pi/2, & \text{if driving north,} \\ \pi & \text{if driving west,} \\ 2\pi/3, & \text{otherwise,} \end{cases} \tag{1}$$

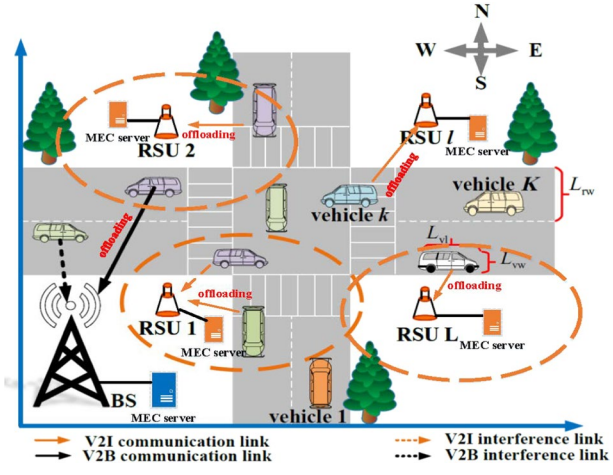


**Fig. 1** The uplink MEC-assisted V2X system

Let $(X_{k,t}, Y_{k,t})$ denote the position of the $k$-th vehicle at the time slot $t$. Thus, the position of the $k$-th vehicle at the next time slot $t'$ are given by

$$X_{k,t'} = X_{k,t} + v_{k,t}\cos\theta_{k,t}\Delta t + \frac{1}{2}\xi_{k,t}\cos\theta_{k,t}(\Delta t)^2, \tag{2}$$

$$Y_{k,t'} = Y_{k,t} + v_{k,t}\sin\theta_{k,t}\Delta t + \frac{1}{2}\xi_{k,t}\sin\theta_{k,t}(\Delta t)^2, \tag{3}$$

where $\Delta t$ is the time interval, and $\xi_{k,t}$ is the driving acceleration of the $k$-th vehicle at the time slot $t$. Thus, the speed of the $k$-th vehicle at the time slot $t'$ can be defined as

$$v_{k,t'} = v_{k,t} + \xi_{k,t}\Delta_t. \tag{4}$$

The time slot $T_k^{\mathrm{D}}$ is the $k$-th vehicle driving out of the intersection area, which can be written as

$$T_k^{\mathrm{D}} = \begin{cases} T_k^{\mathrm{D}}, & \text{if} \quad \theta_{k,t} = 0 \quad \text{and} \quad X_{k,t} \geq L_{\mathrm{rl}}, \\ T_k^{\mathrm{D}}, & \text{if} \quad \theta_{k,t} = \pi/2 \quad \text{and} \quad Y_{k,t} \geq L_{\mathrm{rl}}, \\ T_k^{\mathrm{D}} & \text{if} \quad \theta_{k,t} = \pi \quad \text{and} \quad X_{k,t} \leq 0 \\ T_k^{\mathrm{D}}, & \text{if} \quad \theta_{k,t} = 2\pi/3 \quad \text{and} \quad Y_{k,t} \leq 0, \\ t, & \text{otherwise.} \end{cases} \tag{5}$$

#### *2.1.2 Collision model*

The distance between the $k$-th and $j$-th vehicles at the time slot $t$ is given by

$$d_{k,j,t} = \sqrt{(X_{k,t} - X_{j,t})^2 + (Y_{k,t} - Y_{j,t})^2}. \tag{6}$$

When the driving direction of the $k$-th and $j$-th vehicle is horizontal i.e., $|\theta_k - \theta_j| = 0/\pi$, the collision model is shown in Fig. 2. The maximum horizontal collision distance $d_{\mathrm{pm}} = \sqrt{L_{\mathrm{vl}}^2 + L_{\mathrm{vw}}{}^2}$. Thus, a collision can be detected if $d_{k,j,t} \leq d_{\mathrm{pm}}$. The collision number of the $k$-th and $j$-th $F_k = F_k + 1, \quad F_j = F_j + 1$. When the driving direction of the $k$-th and $j$-th vehicle is vertical i.e., $|\theta_k - \theta_j| = \pi/2$, the collision model is shown in Fig. 3. The maximum vertical collision distance $d_{\mathrm{vm}} = \frac{\sqrt{2}}{2}(L_{\mathrm{vl}} + L_{\mathrm{vw}})$. Thus, a collision can be detected if $d_{k,j,t} \leq d_{\mathrm{vm}}$. The collision number of the $k$-th and $j$-th $F_k = F_k + 1, \quad F_j = F_j + 1$.

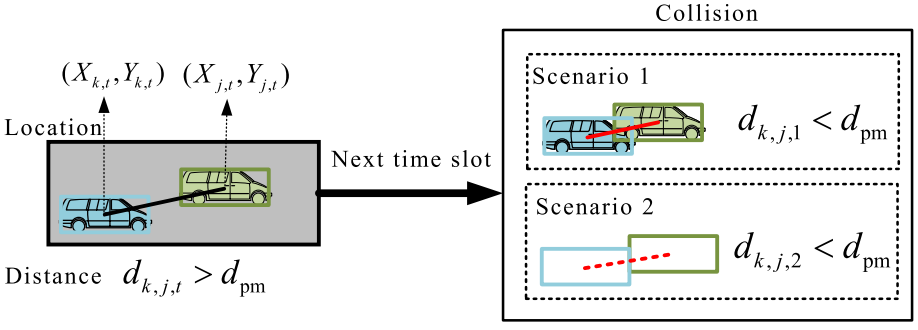


**Fig. 2** The collision model when the driving direction of vehicles is horizontal

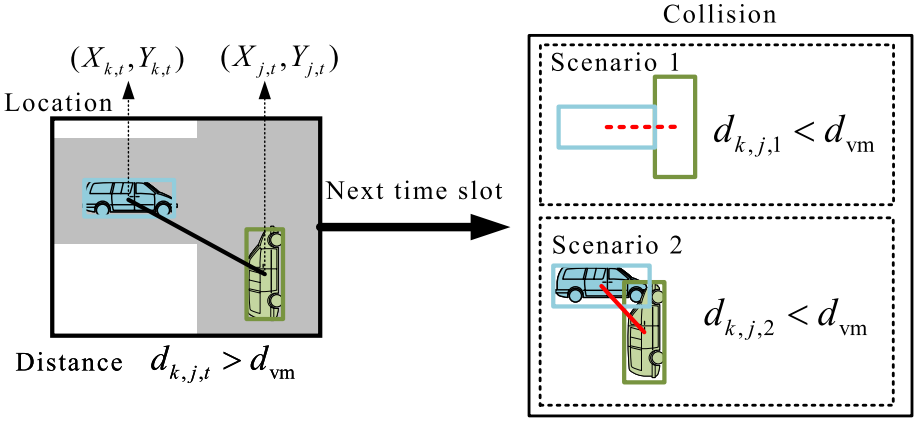


**Fig. 3** The collision model when the driving direction of vehicles is vertical

### 2.2 Channel model

Due to the RSUs and BS equipped with the multiple antennas, the communication interference and computation complexity increase as the number of antennas grows. Thus, the RSUs and BS adopt the ZF receivers to mitigate the communication interference and reduce the computation complexity [25, 28].

#### *2.2.1 Channel model of RSUs with ZF receiver*

For simplicity, the small-scale and large-scale fading at the time slot $t$ between the $k$-th vehicle and the $l$-th RSU with $N^{\mathrm{R}}$-antenna are $\mathrm{h}_{k,l,t}^{\mathrm{R}} \in \mathbb{C}^{N^{\mathrm{R}}}$ and $\beta_{k,l,t}^{\mathrm{R}}$, respectively. The corresponding fading matrix between $K$ vehicles and the $l$-th RSU are $\mathrm{H}_{l,t}^{\mathrm{R}} = [\mathrm{h}_{1,l,t}^{\mathrm{R}}, \mathrm{h}_{2,l,t}^{\mathrm{R}}, \ldots, \mathrm{h}_{K,l,t}^{\mathrm{R}}] \in \mathbb{C}^{N^{\mathrm{R}} \times K}$ and the diagonal matrix $\mathrm{D}_{l,t}^{\mathrm{R}} = \mathrm{diag}[\beta_{1,l,t}^{\mathrm{R}}, \beta_{2,l,t}^{\mathrm{R}}, \ldots, \beta_{K,l,t}^{\mathrm{R}}] \in R^{K \times K}$. Thus, the channel matrix between $K$ vehicles and the $l$-th RSU is $\mathrm{G}_{l,t}^{\mathrm{R}} = \mathrm{H}_{l,t}^{\mathrm{R}} (\mathrm{D}_{l,t}^{\mathrm{R}})^{1/2}$. The communication rate of the $k$-th vehicle associated the $l$-th RSU with ZF receiver is given by

$$R_{k,l,t}^{\mathrm{R}} = B^{\mathrm{V2I}} \log_2 (1 + \frac{p_k}{\sigma_{\mathrm{V2I}}^2 [((\mathrm{G}_{l,t}^{\mathrm{R}})^{\mathrm{H}} \mathrm{G}_{l,t}^{\mathrm{R}})^{-1}]_{k,k}}), \tag{7}$$

where $B^{\mathrm{V2I}}$ is the V2I communication link bandwidth between the $k$-th vehicle and the $l$-th RSU, $p_k$ is the transmit power of the $k$-th vehicle, and $\sigma_{\mathrm{V2I}}^2$ is the additive complex Gaussian white noise power of the V2I communication link, respectively.

#### *2.2.2 Channel model of the BS with ZF receiver*

The small-scale and large-scale fading at the time slot $t$ between the $k$-th vehicle and the BS with $N^{\mathrm{B}}$-antenna are $\mathrm{h}_{k,t}^{\mathrm{B}} \in \mathbb{C}^{N^{\mathrm{B}}}$ and $\beta_{k,t}^{\mathrm{B}}$, respectively. Thus, the small-scale and large-scale fading matrix between $K$ vehicles and the BS are $\mathrm{H}_{t}^{\mathrm{B}} = [\mathrm{h}_{1,t}^{\mathrm{B}}, \mathrm{h}_{2,t}^{\mathrm{B}}, \ldots, \mathrm{h}_{K,t}^{\mathrm{B}}] \in \mathbb{C}^{N^{\mathrm{B}} \times K}$ and the diagonal matrix $\mathrm{D}_{t}^{\mathrm{B}} = \mathrm{diag}[\beta_{1,t}^{\mathrm{B}}, \beta_{2,t}^{\mathrm{B}}, \ldots, \beta_{K,t}^{\mathrm{B}}] \in R^{K \times K}$, respectively. Thus, the channel matrix between $K$ vehicles and the BS is $\mathrm{G}_{t}^{\mathrm{B}} = \mathrm{H}_{t}^{\mathrm{B}} (\mathrm{D}_{t}^{\mathrm{B}})^{1/2} \in \mathbb{C}^{N^{\mathrm{B}} \times K}$. The communication rate of the $k$-th vehicle associated to the BS with ZF receiver is given by

$$R_{k,t}^{\mathrm{B}} = B^{\mathrm{V2B}} \log_2 (1 + \frac{p_k}{\sigma_{\mathrm{V2B}}^2 [((\mathrm{G}_{t}^{\mathrm{B}})^{\mathrm{H}} \mathrm{G}_{t}^{\mathrm{B}})^{-1}]_{k,k}}), \tag{8}$$

where $B^{\mathrm{V2B}}$ and $\sigma^2_{\mathrm{V2B}}$ denote the V2B communication link bandwidth between the *k*-th vehicle and the BS, and the additive complex Gaussian white noise power of the V2B communication link, respectively.

### 2.3 Computation delay and energy consumption

#### *2.3.1 Local computation delay and energy consumption*

For simplicity, let $S_k$ and $\epsilon_k$ denote the data packet size and the offloading ratio of vehicle *k*, respectively. The local computation delay and energy consumption at the vehicle *k* can be written as [30]

$$T^{\mathrm{L}}_{k,t} = \frac{(1-\epsilon_k)S_k}{f^{\mathrm{V}}_k}, \quad E^{\mathrm{L}}_{k,t} = c_0(1-\epsilon_k)S_k(f^{\mathrm{V}}_k)^2, \tag{9}$$

where $f^{\mathrm{V}}_k$ and $c_0$ denote the computation capability of the vehicle *k* and the energy coefficient related to the chip architecture of vehicles, respectively.

#### *2.3.2 RSU computation delay and energy consumption*

When the vehicle *k* offloads the task to the RSU *l*, the communication delay and energy consumption at the time slot *t* are given by

$$T^{\mathrm{O,R}}_{k,l,t} = \frac{\epsilon_k S_k}{R^{\mathrm{R}}_{k,l,t}}, \quad E^{\mathrm{O,R}}_{k,l,t} = p_k T^{\mathrm{O,R}}_{k,l,t}. \tag{10}$$

The execuation delay and the corresponding energy consumption at RSU *l* can be given as

$$T^{\mathrm{E,R}}_{l,t} = \frac{\epsilon_k S_k}{f^{\mathrm{R}}_l}, \quad E^{\mathrm{E,R}}_{l,t} = c_1\epsilon_k S_k(f^{\mathrm{R}}_l)^2, \tag{11}$$

where $c_1$ denotes the energy consumption coefficient related to the chip architecture of RSU, and $f^{\mathrm{R}}_l$ is the processing capability of RSU *l*. Thus, the processing delay and energy consumption of the vehicle *k* offloading to RSUs, are given by

$$T^{\mathrm{R}}_{k,t} = T^{\mathrm{O,R}}_{k,l,t} + T^{\mathrm{E,R}}_{l,t}, \quad E^{\mathrm{R}}_{k,t} = E^{\mathrm{O,R}}_{k,l,t} + E^{\mathrm{E,R}}_{l,t}. \tag{12}$$

#### *2.3.3 BS computation delay and energy consumption*

The communication delay and energy consumption between vehicle *k* and BS are given by

$$T^{\mathrm{O,B}}_{k,t} = \frac{\epsilon_k S_k}{R^{\mathrm{B}}_{k,t}}, \quad E^{\mathrm{O,B}}_{k,t} = p_k T^{\mathrm{O,B}}_{k,t}. \tag{13}$$

Let $f^{\mathrm{B}}$ represent the processing capability of the BS. The execution delay and energy consumption at the BS are given by

$$T^{\mathrm{E,B}}_{t} = \frac{\epsilon_k S_k}{f^{\mathrm{B}}}, \quad E^{\mathrm{E,B}}_{t} = c_2\epsilon_k S_k(f^{\mathrm{B}})^2, \tag{14}$$

where $c_2$ denotes the energy consumption coefficient related to the chip architecture of BS. When the *k*-th vehicle offloads its task to the BS, the processing delay and energy consumption are written as

$$T_{k,t}^{\mathrm{B}} = T_{k,t}^{\mathrm{O,B}} + T_t^{\mathrm{E,B}}, \quad E_{k,t}^{\mathrm{B}} = E_{k,t}^{\mathrm{O,B}} + E_t^{\mathrm{E,B}}. \tag{15}$$

#### *2.3.4 Task processing delay and energy consumption*

At the time slot *t*, when the vehicle *k* associates to the RSU, the task processing consists of the local computation delay $T_{k,t}^{\mathrm{L}}$ and the RSU computation delay $T_{k,t}^{\mathrm{R}}$, while the energy consumption consists of the local computation energy consumption $E_{k,t}^{\mathrm{L}}$ and the RSU computation energy consumption $E_{k,t}^{\mathrm{R}}$. When the vehicle *k* associates to the BS, the processing delay includes both the local computation delay $T_{k,t}^{\mathrm{L}}$ and the BS computation delay $T_{k,t}^{\mathrm{B}}$, while the energy consumption includes the local computation energy consumption $E_{k,t}^{\mathrm{L}}$ and the BS computation energy consumption $E_{k,t}^{\mathrm{B}}$.

For simplicity, the task processing delay and energy consumption of the vehicle *k* at the time slot *t* can be defined as

$$\begin{aligned} T_{k,t}^{\mathrm{C}} &= T_{k,t}^{\mathrm{L}} + (1 - \lfloor \frac{\mu_k}{L+1} \rfloor) T_{k,t}^{\mathrm{R}} + \lfloor \frac{\mu_k}{L+1} \rfloor T_{k,t}^{\mathrm{B}}, \\ E_{k,t}^{\mathrm{C}} &= E_{k,t}^{\mathrm{L}} + (1 - \lfloor \frac{\mu_k}{L+1} \rfloor) E_{k,t}^{\mathrm{R}} + \lfloor \frac{\mu_k}{L+1} \rfloor E_{k,t}^{\mathrm{B}}, \end{aligned} \tag{16}$$

where $\lfloor \cdot \rfloor$ denotes the floor operation and $\mu_k \in \{1, ..., L+1\}$ is the server association. When $\mu_k = l, 1 \leq l \leq L$ or $\mu_k = L+1$ denote that the vehicle *k* offloads the task to RSU *l* or the BS for edge computing, respectively. In other words, only one edge server associate to the vehicle *k* for offloading the task.

### 2.4 System delay minimization problem

Althougth offloading the computation-intensive task the BS or RSU servers can reduce the processing delay by optimizing the driving acceleration, it may cause the increasing of the driving delay for the multiple vehicle. Thus, the system delays of *K* vehicles consist of the task processing delay $\sum_{k=1}^{K} \sum_{t=1}^{T_k^{\mathrm{D}}} T_{k,t}^{\mathrm{C}}$ and driving delay $\sum_{k=1}^{K} T_k^{\mathrm{D}} \Delta_t$.

In order to reduce the system delay and guarantee the road safety, this work formulates the system delay minimization problem under the delay, energy consumption and collision constraints.

$$(\mathrm{P1}): \min_{\epsilon_k, \mu_k, \xi_k} \sum_{k=1}^{K} \sum_{t=1}^{T_k^{\mathrm{D}}} T_{k,t}^{\mathrm{C}} + T_k^{\mathrm{D}} \Delta_t \tag{17}$$

$$s.t. \epsilon_k \in [0,1], k \in \mathcal{K}, \tag{17.a}$$

$$\mu_k = \{1, 2, \cdots, L+1\}, k \in \mathcal{K}, \tag{17.b}$$

$$\xi_k \in [\xi_{\min}, \xi_{\max}], k \in \mathcal{K}, \tag{17.c}$$

$$v_k \in [v_{\min}, v_{\max}], k \in \mathcal{K}, \tag{17.d}$$

$$F_k = 0, k \in \mathcal{K}, \tag{17.e}$$

$$T_{k,t}^{\mathrm{C}} \leq T_{\max}^{\mathrm{C}}, k \in \mathcal{K}, \tag{17.f}$$

$$E_{k,t}^{\mathrm{C}} \leq E_{\max}^{\mathrm{C}}, k \in \mathcal{K}, \tag{17.g}$$

$$T_{k}^{\mathrm{D}} \leq T_{\max}^{\mathrm{D}}, k \in \mathcal{K}, \tag{17.h}$$

where $\mathcal{K} = \{1, 2, ..., K\}$ is the set of vehicles in the V2X system. The constraint (17a) is the offloading ratio of the vehicles, where $\epsilon_k = 0$ denotes the local processing. The constraint (17b) represents that the vehicle $k$ associates to the BS edge server, i.e., $\mu_k = L + 1$, or RSU $l$ edge server, i.e., $\mu_k = l$. The constraints (17c) and (17d) denote the range of the driving acceleration and speed to characterize the intersection scenario, respectively. The constraint (17e) is the collision number by computing the vehicles' distance to guarantee the road safety. In addition, the constraints (17f–h) are the maximum processing delay $T_{\max}^{\mathrm{C}}$, the maximum energy consumption $E_{\max}^{\mathrm{C}}$ and the maximum interval delay $T_{\max}^{\mathrm{D}}$, respectively.

Due to the non-convex objective function and the coupling between variables $\mu_k$ and $\epsilon_k$ in (9) and (16), the traditional convex optimization methods are difficult to solve the problem (P1) efficiently. To address this optimization problem, an intelligent driving algorithm based on DQN is proposed to jointly optimize server association, offloading ratio and driving acceleration.

## 3 Proposed DQN algorithm for solving problem (P1)

DRL algorithm is an intelligent learning method, which enables the agents to learn dynamically and make the optimal real-time decisions by interacting with the environment without requiring the prior information. In this work, the system delay optimization problem (P1) is non-convex and hard to solve due to the coupling of the server association, offloading ratio and driving acceleration. Thus, the joint offloading, association and driving acceleration scheme based on DQN is adopted to solve the complex multiple action spaces problem.

Q-learning algorithm seeks to determine the optimal policy through maximum Q-value estimation for state-action pairs recorded in the Q-table. The agent updates the Q-table during the learning process to record historical experiences. It is difficult for the Q-table with a fixed size to store all state-action pairs when the environment has a large number of states or the agents have large action spaces. Besides, the algorithm complexity of Q-learning is related to the dimension of the state space, action space and the number of the agents [25]. DQN is proposed to address the limitations of traditional Q-learning algorithm and adopts the experience replay buffer to overcome instability caused by using a non-linear function to approximate the Q-value [31]. Thus, the proposed intelligent offloading, association and driving acceleration scheme based on DQN algorithm can solve the delay minimization problem by optimizing the server association, offloading ratio and driving acceleration jointly.

The optimization problem (P1) can be modeled as a Markov decision process. In this work, vehicles work as the agents that intelligently select the best actions and interact with the V2X system's environment.

1) *state* $s_{k,t}$: at the time slot $t$, the state space of vehicle $k$ comprises the data packet size $S_K$, position $(X_{k,t}, Y_{k,t})$ and the system task processing delay.

2) *action* $a_{k,t}$: at the time slot $t$, the action space of agents is defined as $\mathrm{a}_{k,t} = \{A_\epsilon, A_\mu, A_\xi\}$. The offloading ratio action spaces is $A_\epsilon = \{\epsilon_k = [\frac{1}{d_\epsilon}, \frac{2}{d_\epsilon}, \frac{3}{d_\epsilon}, \ldots, 1]\}$, where $d_\epsilon$ is the length of the offloading ratio action space. The server association and acceleration action spaces are $A_\mu = \{\mu_k = [1, 2, \cdots, L+1]\}$, and $A_\xi = \{\xi_k \in \{-9, -2, 0, 2\}\}$ [27], respectively.
3) *reward* $r_{k,t}$: The aim of the proposed algorithm is to derive the optimal policy by estimating the maximum Q-values, thus, the reward has a great influence on the action selection. In order to reduce the system delay while avoiding collisions, the reward of the $k$-th vehicle at the time slot $t$ is modeled as

$$r_{k,t} = \begin{cases} -T^{\mathrm{C}}_{k,t}, & (17.f) \quad \text{not satisfied}, \\ -E^{\mathrm{C}}_{k,t}, & (17.g) \quad \text{not satisfied}, \\ -v_k, & (17.d) \quad \text{not satisfied}, \\ -F_k, & (17.e) \quad \text{not satisfied}, \\ b/(T^{\mathrm{C}}_{k,t} + T^{\mathrm{D}}_{k} \Delta_t), & \text{otherwise}, \end{cases} \tag{18}$$

where $b$ is a positive number. If all constraints (17d–g) are satisfied, the $k$-th vehicle can obtain the positive reward, which is negatively related with the sum of the task processing delay and vehicle driving delay. Otherwise, it obtains the negative reward which is related to the task processing delay, the energy consumption, the vehicle speed and the collision number, respectively.

The framework of the intelligent offloading, association and driving algorithm based on DQN is shown in Fig. 4. It contains an experience replay buffer and estimate and target neural networks. at the time slot $t$, the $k$-th vehicle acts as an agent, choose action $\mathrm{a}_{k,t}$ based on the current environment state $\mathrm{s}_{k,t}$ and interact with the environment, observes the next state $\mathrm{s}^{'}_{k,t}$ and obtains the reward $r_{k,t}$. $< \mathrm{s}_{k,t}, \mathrm{a}_{k,t}, r_{k,t}, \mathrm{s}^{'}_{k,t} >$ is stored in the experience replay buffer. During the exploration period, DQN randomly selecting a mini-batch of samples from it and trains the agents. Specifically, the estimate and target network takes $\mathrm{s}_{k,t}, \mathrm{a}_{k,t}$ and $r_{k,t}, \mathrm{s}^{'}_{k,t}$ from each data sample, respectively. The estimate

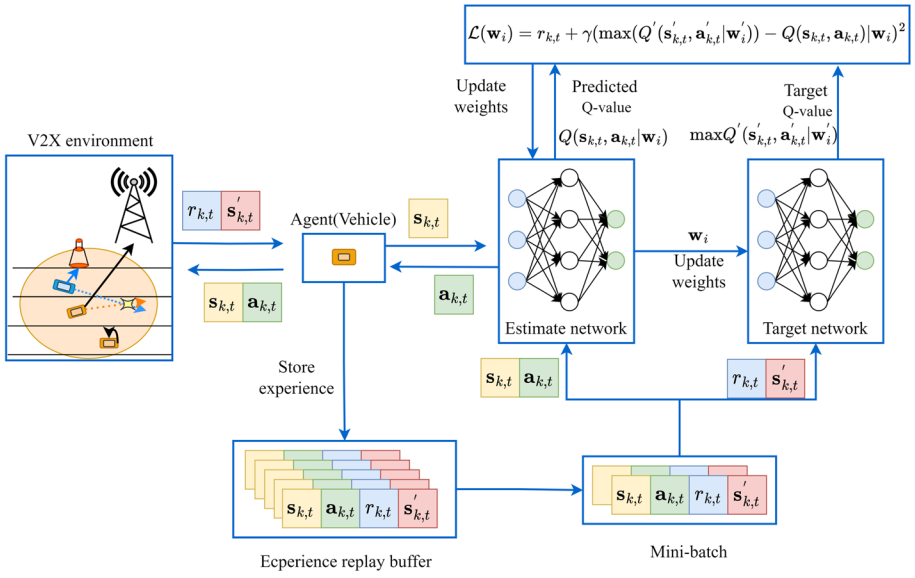


**Fig. 4** The framework of DQN-based intelligent offloading, association and driving algorithm

network computes the predicted Q-value $Q(\mathrm{s}_{k,t}, \mathrm{a}_{k,t})$ for the action $\mathrm{a}_{k,t}$. Meanwhile, the target network computes the maximum Q-value, $\max(Q^{'}(\mathrm{s}^{'}_{k,t}, \mathrm{a}^{'}_{k,t}))$, from all the feasible actions that can be executed in state $\mathrm{s}^{'}_{k,t}$. Updating Q-value based on the bellman equation [24, 32]

$$\begin{aligned} &Q(\mathrm{s}_{k,t}, \mathrm{a}_{k,t}) = \\ &\quad (1-\alpha)Q(\mathrm{s}_{k,t}, \mathrm{a}_{k,t}) + \alpha(r_{k,t} + \gamma \max(Q^{'}(\mathrm{s}^{'}_{k,t}, \mathrm{a}^{'}_{k,t}))), \end{aligned} \tag{19}$$

where $\alpha$ and $\gamma$ denote the learning rate and discount factor, respectively. Then, the neural networks adjust the network parameters depending on the loss function, which can be written as [32]

$$\mathfrak{I}(\mathrm{w}_i) = r_{k,t} + \gamma(\max(Q^{'}(\mathrm{s}^{'}_{k,t}, \mathrm{a}^{'}_{k,t}|\mathrm{w}^{'}_i)) - Q(\mathrm{s}_{k,t}, \mathrm{a}_{k,t})|\mathrm{w}_i)^2, \tag{20}$$

where $\mathrm{w}_i$ and $\mathrm{w}^{'}_i$ are the weights parameters of the estimate and target networks during the $i$-th round training. After many iterations, the agent can choose better actions consist of the server association, offloading ratio, and driving acceleration. The detailed description of the proposed scheme is shown in Algorithm 1.

**Initial** The location of the BS and RSUs. The location and speed of $K$ vehicles. Learning rate $\alpha$ and reward discount factor $\gamma$. The estimate and target networks for all vehicles.
**for** each iteration episode **do**
The $k$-th vehicle observes the environment state $\boldsymbol{s}_{k,t}$ and selects the action $\mathbf{a}_{k,t}$ from the action space $\{A_\varepsilon, A_\mu, A_\xi\}$. Compute the reward based on (18) and store $< \boldsymbol{s}_{k,t}, \mathbf{a}_{k,t}, r_{k,t}, \boldsymbol{s}^{'}_{k,t} >$ in the experience replay buffer.
**for** each training step **do**
Randomly select the mini-batch data from the experience replay buffer. Compute the predicted Q-value $Q(\boldsymbol{s}_{k,t}, \mathbf{a}_{k,t})$ based on the estimate network and compute the target Q-value $\max(Q^{'}(\mathbf{s}^{'}_{k,t}, \mathbf{a}^{'}_{k,t}))$ based on the target network, respectively.
Compute the loss function in (20). Update the Q-value based on (19) and the target network parameter weights.
**end for**
**end for**
Save the action $\mathbf{a}_{k,t}$ and the delay $T^{\mathrm{C}}_{k,t}$.

**Algorithm 1** DQN-based offloading, association and driving scheme

## 4 Simulation results

To verify the effectiveness of the proposed scheme, the numerical results are provided in this section. This work adopts an intersection road with the width of 300 m and length of 600 m. The number of vehicles and RSUs are 6 and 4, respectively. The simulation environment is Python 3.7.4 and Pytorch 1.12.1. Unless otherwise stated, some simulation parameters are shown in Table 1. Additionally, the Q-learning and local computing schemes are adopted to compared with the proposed algorithm. Specifically, the local computing scheme means that vehicles compute all tasks locally, i.e., the offloading ratio $\epsilon_k = 0, k \in \mathcal{K}$.

Figure 5 shows the convergence performance of proposed algorithms under different learning rates. In the case of learning rate $\alpha = 0.001$ and $\alpha = 0.01$, it can be found that the system delay of the proposed scheme decreases and finally stays stable as the training episode increases. When the learning rate $\alpha = 0.01$, the proposed algorithm converges

**Table 1** The simulation parameters

| Symbol | Description | Value |
|---|---|---|
| $L_{\mathrm{rl}}$ | The road length | 600 m |
| $L_{\mathrm{rw}}$ | The road width | 3.75 m |
| $L_{\mathrm{vl}}$ | The vehicle length | 4 m |
| $L_{\mathrm{vw}}$ | The vehicle width | 2 m |
| $N^{\mathrm{B}}$ | The number of BS antenna | 4 |
| $f^{\mathrm{B}}$ | BS processing capability | 10 Mbits/s |
| $N^{\mathrm{R}}$ | The number of RSU antenna | 2 |
| $f^{\mathrm{R}}$ | RSU processing capability | 5 Mbits/s |
| $D_k$ | Task packet size | 2 Mbits |
| $p$ | Vehicle transmit power | 23 dBm |
| $f^{\mathrm{V}}$ | Vehicle processing capability | 1 Mbits/s |
| $v_{\max}$ | Maximum speed | 60 Km/h |
| $v_{\min}$ | Minimum speed | 30 Km/h |
| $T_{\max}$ | Maximum task processing delay | 1 s |
| $\mathrm{B}^{\mathrm{V2I}}$ | Bandwidth of the V2I communication link | 1 MHz |
| $\mathrm{B}^{\mathrm{V2B}}$ | Bandwidth of the V2B communication link | 5 MHz |
| $d_\epsilon$ | Length of the offloading ratio action space | 5 |
| $\alpha$ | Learning rate | 0.01 |
| $N_{\mathrm{s}}$ | Training steps in each episode | 600 |
| $N_{\mathrm{e}}$ | Iteration episodes | 100 |

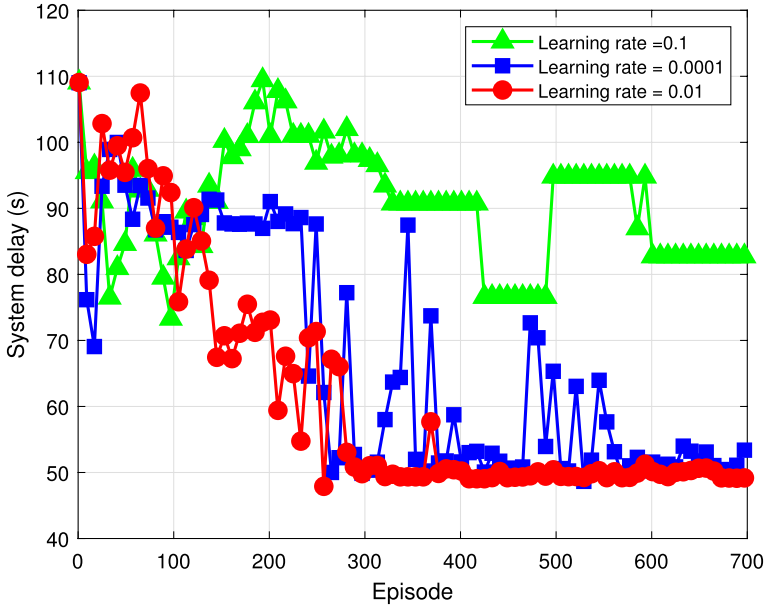


**Fig. 5** System delay versus training episode under different learning rate

more rapidly and stably compared with $\alpha = 0.001$. When the learning rate $\alpha = 0.1$, the system delay is difficult to stay stable as the training episode increases. Thus, the learning rate $\alpha = 0.01$ is employed to train the vehicles in selecting the optimal server association, offloading ratio and driving acceleration for system delay minimization.

The effect of training steps on the system delay of each vehicle of the proposed scheme with $\alpha = 0.01$ is depicted in Fig. 6. It can be observed that the delay of each vehicle is smaller than 1 s and stays stable after 30,000 steps. Figure 7 shows the influence of the number of training episodes on the speed of each vehicle. We can find that the speed of

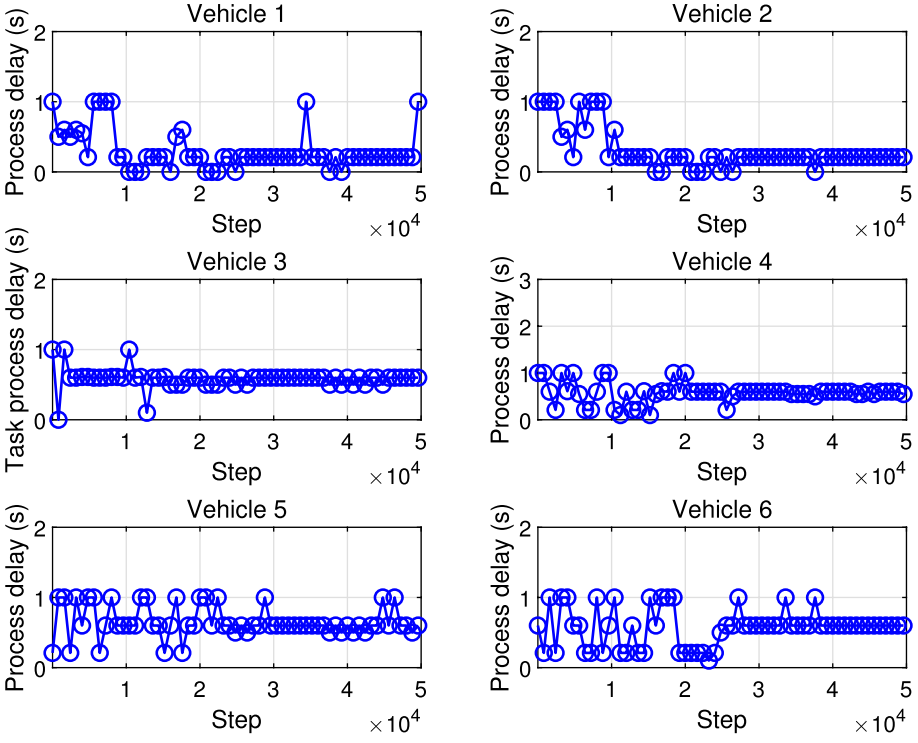


**Fig. 6** System delay versus training step with $\alpha = 0.01$

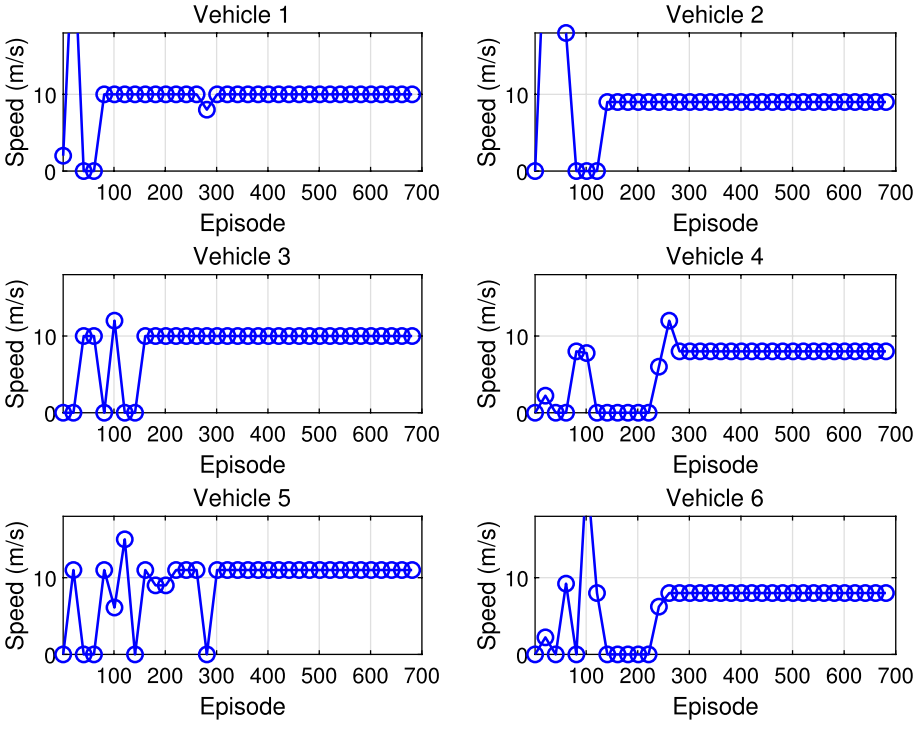


**Fig. 7** Vehicle speed versus training episode with $\alpha = 0.01$

each vehicle is less than the maximum speed $v_{\max}$ and more than the minimum speed $v_{\min}$. Thus, the proposed scheme can reduce the system delay while satisfying the speed constraint.

Figure 8 shows that the convergence of system delay of three schemes. It can be found that, the system delays of three schemes decrease and finally becomes stable. The proposed DQN optimization scheme can reduce 16.44% system delay, and more fast and stable convergence compared with Q-learning scheme. This is because that the large Q-table exists in the Q-learning scheme leading to the weaken ability to find the optimal speed and states. The local computing scheme only optimizes the MEC server association, ignores the impact of offloading ratio. Thus, the proposed scheme can reduce the system delay by 26.64% compared with the local computing scheme.

The influence of the packet size on the system delays of three schemes are shown in Fig. 9. As the packet size increases, the system delays of three schemes all increase. It originates that vehicles need more time to process the task when the packet size increases. Besides, the proposed scheme can achieve significant performance gain in reducing the system delay compared the Q-learning and local computing schemes. Specifically, when the packet size is 0.6Mbits, the system delay of the proposed scheme reduces by 17.70% and 20.45% compared with Q-learning and local computing schemes, respectively. When the packet size is 1.4Mbits, the system delay of the proposed scheme reduces by 13.89% and 32.76% compared with Q-learning and local computing schemes, respectively. Thus, the proposed scheme can reduce the system delay compared with the benchmark schemes under different packet sizes.

The effect of the number of vehicles on the system delay for three schemes is depicted in Fig. 10. We can find that the system delays of three schemes increase as the number of vehicles increases. On the one hand, the increasing number of vehicles causes the packet size of the system increases, the increasing number of vehicles can reduce the communication rate because more vehicles must offload the task the long-distance BS. Besides, the number of vehicles increases may lead to more vehicle collisions. In order

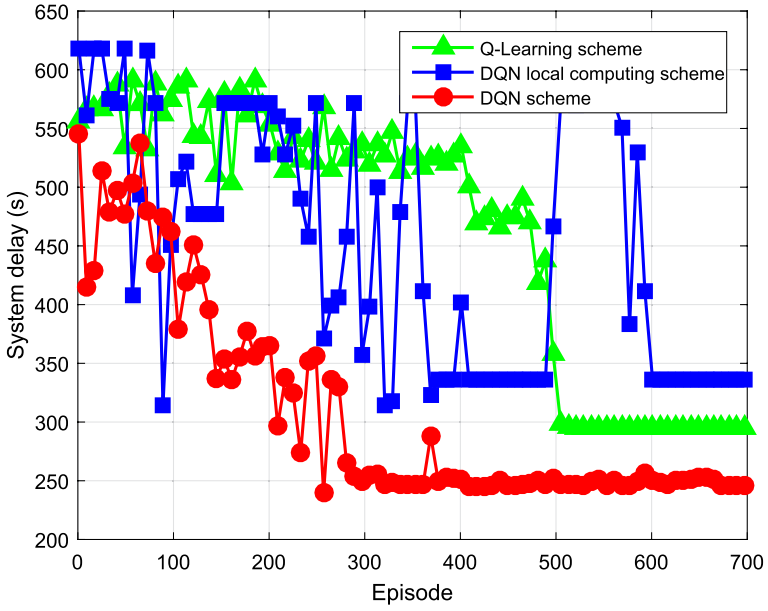


**Fig. 8** System delay versus training episode under different schemes

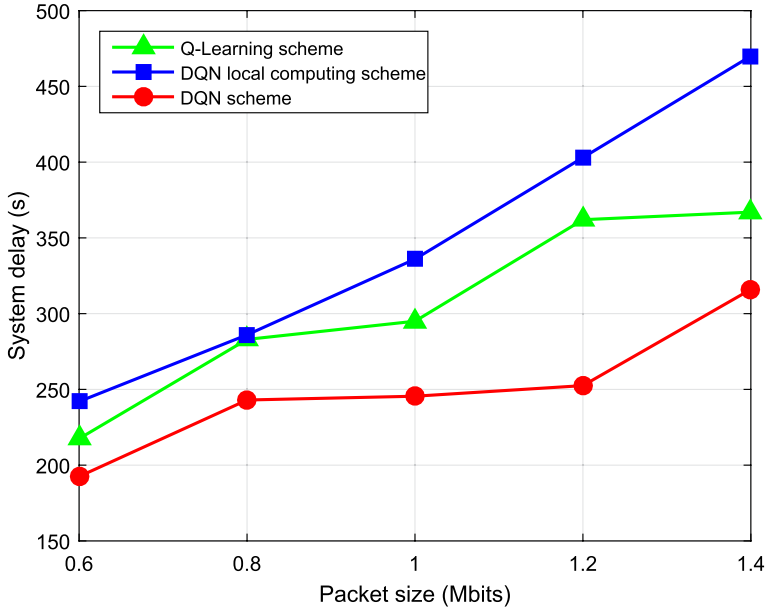


**Fig. 9** System delay versus packet size under different schemes

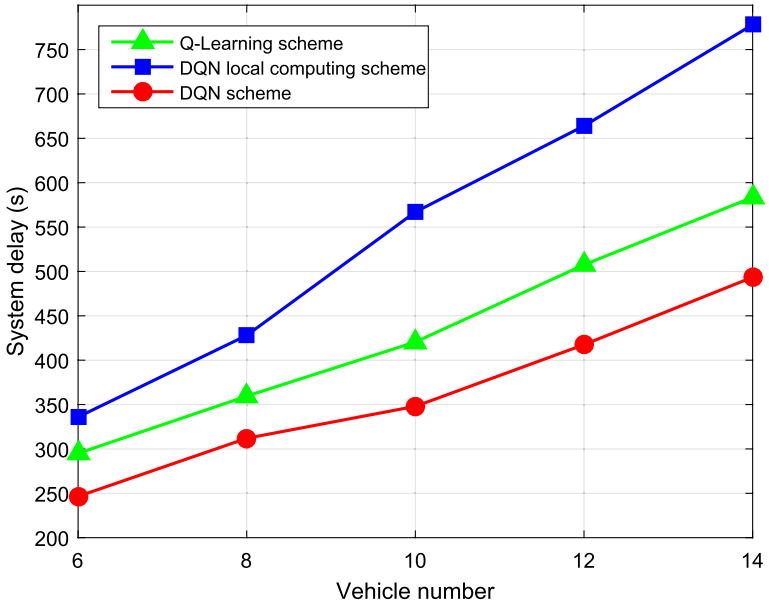


**Fig. 10** System delay versus number of vehicles under different schemes

to satisfy the collision constraint, vehicles will drive at a lower speed, thereby the driving delay increases. When the number of vehicles is 10, the system delay of the proposed scheme reduces by 17.24% and 38.62% compared with Q-learning and local computing schemes, respectively. When the number of vehicles is 14, the system delay of the proposed scheme reduces by 13.27% and 29.82% compared with Q-learning and local computing schemes, respectively. Thus, the proposed scheme can reduce the system delay compared with the benchmark schemes under different number of vehicles settings.

Figure 11 shows the relationship between the road length and the system delays of three schemes. It can be seen that as the road length increases, the system delay

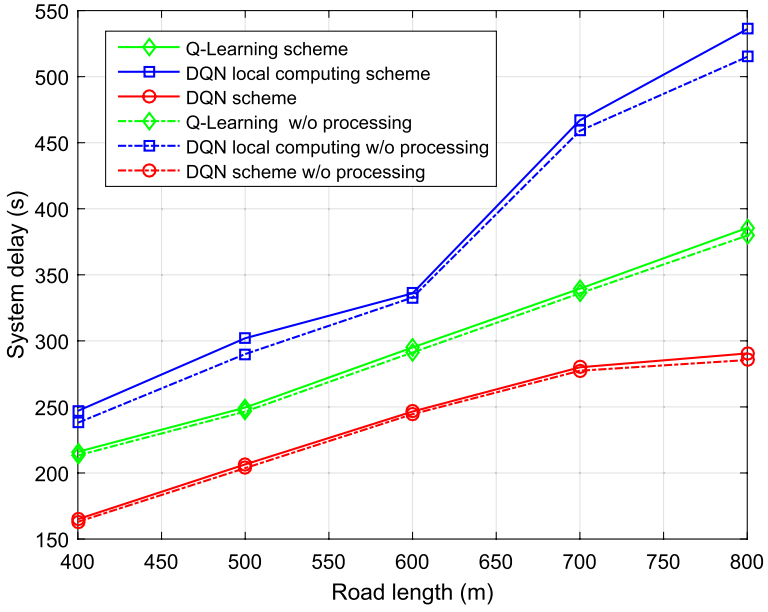


**Fig. 11** System delay versus road length under different schemes

increases under different schemes. This is because that the increasing road length causes to more time driving out of the road and make the driving delay increasing. Thus, the driving delay and task processing delay increase. When the road length is 400 m, the proposed scheme reduces the system delay by 23.61% and 33.20% compared with the Q-learning and local computing schemes, respectively. When the road length is 800 m, the proposed scheme reduces the system delay by 24.64% and 45.80% compared with the Q-learning and local computing schemes, respectively. In addition, when the road length is 400 m, the average processing delays of Q-learning, local computing and proposed DQN schemes are 0.45s, 1.5s and 0.33s, respectively. When the road length is 800 m, the average processing delays of Q-learning, local computing and proposed DQN schemes are 0.97s, 3.5s and 0.82s. The proposed scheme can reduce the average processing delay compared with the benchmark schemes.

The influence of the number of vehicles on the collision number is shown in Fig. 12 after 700 episodes. We can observe that the collision number of three schemes increases as the number of vehicles increases. When the number of vehicles is smaller than 10, the Q-learning scheme outperforms the proposed scheme. While the collision number of Q-learning is significant increasing compared with the proposed scheme.

## 5 Conclusions

This work jointly optimized the task offloading, server association and driving acceleration in the MEC-assisted multi-vehicle V2X systems. To minimize the system delay and avoid vehicles collision, the system delay minimization problem was formulated under the energy consumption and vehicle collision constraints. Due to the coupling between the server association, offloading ratio and driving acceleration, the optimization problem is non-convex and hard to solve. Thus, we proposed DQN-based algorithm to solve the problem. Based on the reward function related to the system delay and collision number, vehicles were trained to choose better actions. Simulation results show that the proposed algorithm reduces the system delay while satisfying the task processing delay

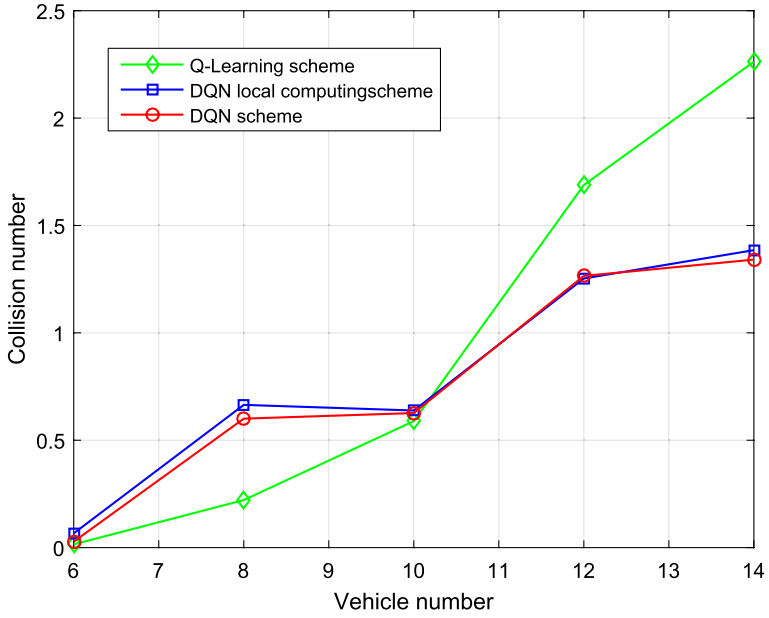


**Fig. 12** Collision number versus the number of vehicles under different schemes

and the collision constraints. Meanwhile it had better performance compared with the Q-learning and local computing schemes under different number of vehicles and road lengths. In the future, the intelligent scheme can be exploited in the scenario of the RSU with multiple servers.

**Acknowledgements**
The authors would like to thank the anonymous reviewers for their comments to improve the quality of the paper.

**Author contributions**
XML, WWD and XHZ conceived of the presented idea and drafted the manuscript. WWD and CWY conducted the experiments. XML, CWY and YFP modified the manuscript and supervised this work. All the authors have read and approved the final manuscript.

**Funding**
The work was supported by Key Laboratory of Pattern Recognition and Intelligent Information Processing, Institutions of Higher Education of Sichuan Province, Chengdu University, China (Grant No.MMSB-2025-09).

**Data availability**
Data sharing is not applicable to this article as no datasets were generated or analysed during the current study.

## Declarations

**Ethics approval and consent to participate**
No individual consent was required as the study did not involve direct human participation or collection of personal data.

**Consent for publication**
Not applicable.

**Competing interests**
The authors declare no Conflict of interest.

Received: 23 August 2025 / Accepted: 26 November 2025
Published online: 09 December 2025

**References**

1. Li G, Lai C, Lu R, Zheng D. SecCDV: a security reference architecture for Cybertwin-Driven 6G V2X. IEEE Trans Veh Technol. 2022;71(5):4535–50.
2. Noor-A-Rahim M, Liu Z, Lee H, Khyam MO, He J, Pesch D, et al. 6G for vehicle-to-everything (V2X) communications: enabling technologies, challenges, and opportunities. Proc IEEE. 2022;110(6):712–34.

3. Clancy J, Mullins D, Deegan B, Horgan J, Ward E, Eising C, et al. Wireless access for V2X communications: research, challenges and opportunities. IEEE Commun Surv Tutorials. 2024;26(3):2082–119.
4. Xu Z, Zhou L, Chi-Kin Chau S, Liang W, Xia Q, Zhou P, Collaborate or separate? Distributed service caching in mobile edge clouds. In: IEEE conference on computer communications. 2020;2066–2075.
5. Siriwardhana Y, Porambage P, Liyanage M, Ylianttila M. A survey on mobile augmented reality with 5G mobile edge computing: architectures, applications, and technical aspects. IEEE Commun Surv Tutor. 2021;23(2):1160–92.
6. Jiang H, Dai X, Xiao Z, Iyengar A. Joint task offloading and resource allocation for energy-constrained mobile edge computing. IEEE Trans Mob Comput. 2023;22(7):4000–15.
7. Andrawes A, Nordin R, Albataineh Z, Alsharif MH. Sustainable delay minimization strategy for mobile edge computing offloading under different network scenarios. Sustainability. 2021;13(21):12112.
8. Mohammed F, Pan Z, Haithm A, Qasem Z. Optimizing end-to-end latency in C-V2X networks: a novel FD-RAN and MEC integration approach, Veh Commun. 2025;100955
9. Liu Y, Xu Y, Zhang H, Li Y, Yuen C. Mode selection and resource allocation for MEC-assisted v2x networks under limited energy and bandwidth constraints. IEEE Trans Intell Transp Syst, 2025
10. Sabella D, Lei M. AI and sensor fusion on roadside MEC: standards and implementations for V2X. IEEE Commun Stand Mag. 2025;9(2):80–7.
11. Li Y, Jiang C. Distributed task offloading strategy to low load base stations in mobile edge computing environment. Comput Commun. 2020;164:240–8.
12. Wang Y, Lang P, Tian D, Zhou J, Duan X, Cao Y, et al. A game-based computation offloading method in vehicular multiaccess edge computing networks. IEEE Internet Things J. 2020;7(6):4987–96.
13. Dai X, Xiao Z, Jiang H, Alazab M, Lui S, Dustdar S, et al. Task co-offloading for D2D-assisted mobile edge computing in industrial internet of things. IEEE Trans Industr Inf. 2023;19(1):480–90.
14. Annu, Rajalakshmi P. Towards 6G V2X sidelink: survey of resource allocation-mathematical formulations, challenges, and proposed solutions. IEEE Open J Veh Technol. 2024;5:344–83.
15. Tan L, Kuang Z, Zhao L, Liu A. Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing. IEEE Trans Wirel Commun. 2022;21(3):1960–72.
16. Yadav R, Zhang W, Kaiwartya O, Song H, Yu S. Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing. IEEE Trans Veh Technol. 2020;69(12):14198–211.
17. Li H, Xu H, Zhou C, Lv X, Han Z. Joint optimization strategy of computation offloading and resource allocation in multiaccess edge computing environment. IEEE Trans Veh Technol. 2020;69(9):10214–26.
18. Fu Y, Li C, Yu FR, Luan TH, Zhang Y. A survey of driving safety with sensing, vehicular communications, and artificial intelligence-based collision avoidance. IEEE Trans Intell Transp Syst. 2022;23(7):6142–63.
19. Tan J, Li H, Xia Q. The study of trajectory prediction in V2X collision warning based on pruned SR-LSTM, In: International conference on computer science, electronic information engineering and intelligent control technology (CEI), 2024; 620–625.
20. Deng R, Di B, Song L. Cooperative collision avoidance for overtaking maneuvers in cellular V2X-based autonomous driving. IEEE Trans Veh Technol. 2019;68(5):4434–46.
21. Bachmann M, Morold M, David K. On the required movement recognition accuracy in cooperative VRU collision avoidance systems. IEEE Trans Intell Transp Syst. 2021;22(3):1708–17.
22. Ye F, Cheng X, Wang P, Chan C-Y, Zhang J. Automated lane change strategy using proximal policy optimization-based deep reinforcement learning. In: IEEE intelligent vehicles symposium (IV). 2020;2020:1746–52.
23. Lin Y, Zhang Z, Huang Y, Li J, Shu F, Hanzo L. Heterogeneous user-centric cluster migration improves the connectivity-handover trade-off in vehicular networks. IEEE Trans Veh Technol. 2020;69(12):16027–43.
24. Li X, Li J, Yin B, Yan J, Fang Y, Age of information optimization in UAV-enabled intelligent transportation system via deep reinforcement learning. In: IEEE vehicular technology conference (VTC-Fall). 2022; 1–5.
25. Duan W, Li X, Huang Y, Cao H, Zhang X. Multi-agent-deep-reinforcement-learning-enabled offloading scheme for energy minimization in vehicle-to-everything communication systems. Electronics. 2024;13(3):663–81.
26. Ye F, Cheng X, Wang P, Chan C-Y, Zhang J. Automated lane change strategy using proximal policy optimization-based deep reinforcement learning. In: IEEE intelligent vehicles symposium (IV), 2020; 1746–1752.
27. Hoel C-J, Wolff K, Laine L. Automated speed and lane change decision making using deep reinforcement learning. In: International conference on intelligent transportation systems (ITSC), 2018; 2148–2155.
28. Xie H, Liu H, Chen H, Feng S, Wei Z, Zeng Y. Efficient multi-user resource allocation for urban vehicular edge computing: a hybrid architecture matching approach. IEEE Trans Veh Technol. 2025;74(1):1811–6.
29. Cui Y, Zhang D, Zhang T, Zhang J. A novel offloading scheduling method for mobile application in mobile edge computing. IEEE Trans Mob Comput. 2022;28(2):2345–63.
30. Zhang Y, Dong X, Zhao Y. Decentralized computation offloading over wireless-powered mobile-edge computing networks. In: IEEE international conference on artificial intelligence and information systems (ICAIIS). 2020; 137–140.
31. Mao Q, Hu F, Hao Q. Deep learning for intelligent wireless networks: a comprehensive survey. IEEE Commun Surv Tutor. 2018;20(4):2595–621.
32. Li X, Zhang X, Li J, Luo F, Huang Y, Zhang X. Blocklength allocation and power control in UAV-assisted URLLC system via multi-agent deep reinforcement learning. Int J Comput Intell Syst. 2024;17(1):138–51.


## Publisher's Note